

DevSecOps para leigos

Um guia didático e prático sobre os ciclos e ferramentas dessa cultura baseada no livro "Learning DevSecOps" de Steve Suehring.

Apresentação por Marlon Bastida

O que é DevSecOps?

A união perfeita: integrando **Desenvolvimento (Dev)**, **Segurança (Sec)** e **Operações (Ops)** desde o primeiro dia de um projeto de software, eliminando silos(barreiras) estruturais.

O que é DevSecOps?

A União Perfeita

Integrando Desenvolvimento (Dev), Segurança (Sec) e Operações (Ops) desde o primeiro dia de um projeto de software, eliminando silos estruturais.

A Jornada DevSecOps

O DevSecOps é representado por um ciclo contínuo com 8 fases principais:

1. **Plan** (Planejamento)
2. **Code** (Desenvolvimento)
3. **Build** (Construção)
4. **Test** (Teste)
5. **Release** (Liberação)
6. **Deploy** (Implantação)
7. **Operate** (Operação)
8. **Monitor** (Monitoramento)

Fase 1: Planeamento (Plan)

Alinhamento de Ideias

A equipa define o que será construído. Requisitos de negócio e segurança são discutidos juntos, mapeando riscos antes de qualquer código ser escrito.

Há um "fit" natural com Metodologias Ágeis (Scrum, Kanban). O foco é na velocidade e redução de desperdícios.

Ferramentas de Colaboração

Jira

Gestão ágil de projetos e rastreio de tarefas e sprints.

Slack

Comunicação instantânea, integrando alertas e automatizações.

Microsoft Teams

Colaboração unificada, permitindo alinhamento em tempo real do planeamento.

Fase 2: Desenvolvimento (Code)

Foco

Transformar requisitos em código seguro e rastreável.

Práticas Recomendadas

- **Codificação Segura:** Aplicar padrões desde a primeira linha.
- **Controlo de Versões:** Uso de Git para colaboração e histórico.
- **Linting Automático:** Ferramentas para detetar falhas de sintaxe em tempo real.

IDEs de Desenvolvimento

A sua "Oficina" de Código

IDEs (Ambientes de Desenvolvimento Integrado) são programas que funcionam como uma oficina completa. Eles avisam sobre erros de sintaxe, oferecem plugins de segurança (linting) e formatam o texto automaticamente enquanto escreve, economizando horas de trabalho braçal.

Ferramentas Populares

- **VS Code:** Leve e moderno. É o "canivete suíço" da programação hoje em dia, permitindo instalar milhares de extensões que ajudam a encontrar falhas de segurança instantaneamente.
- **Eclipse:** Robusto e tradicional. Funciona como uma grande fábrica pesada, sendo muito utilizado por empresas corporativas gigantes para programar na linguagem Java.
- **Antigravity:** (Um famoso "easter egg" divertido da linguagem Python). Ilustra a ideia moderna de que as ferramentas tentam fazer o trabalho pesado de forma tão "mágica" que parece que está a flutuar!

Repositórios e Versões

A "Máquina do Tempo" do Software

Estes ecossistemas atuam como o coração da gestão do código. Eles guardam cada alteração (versões) e usam ramificações (branches) para que várias pessoas programem funcionalidades ao mesmo tempo sem quebrar o código principal.

Principais Plataformas

- **Git:** O "Motor". É a ferramenta invisível instalada no computador que tira as "fotografias" de cada alteração.
- **GitHub:** A "Rede Social". Onde o mundo guarda o código Git na nuvem. É a maior montra de projetos de código aberto.
- **GitLab:** A "Fábrica Integrada". Além de guardar o código, brilha no DevSecOps por já vir com motores de testes e segurança (CI/CD) embutidos de fábrica.
- **Bitbucket:** A "Opção Corporativa". Da empresa Atlassian, integra-se perfeitamente com o Jira nas empresas.

A Segurança na Prática: Exemplo Snyk

O "Corretor Ortográfico"

Imagine escrever um e-mail e o computador sublinhar palavras erradas a vermelho. O Snyk faz isso para falhas de segurança enquanto programa!

No Dia a Dia do Programador

- **Análise de Lego:** Verifica se as bibliotecas prontas da internet (bloquinhos) não têm defeitos de fábrica.
- **Alerta em Tempo Real:** Avisa o programador dentro da IDE (como o VS Code).
- **Correção Automática:** Famoso por dar a solução, oferecendo um botão "Corrigir" para trocar a peça vulnerável instantaneamente.

Fase 3: Construção (Build)

A magia da automatização acontece aqui: transformando o código-fonte legível por humanos em pacotes executáveis de software de forma limpa, auditável e reproduzível.

Ferramentas de Construção e Segurança de Imagem

Maven / Gradle

Build. Ferramentas robustas que automatizam a compilação do código e a gestão complexa de dependências para pipelines.

SonarQube

Qualidade. Realiza análises estáticas (SAST) inspecionando o código em busca de bugs e vulnerabilidades antes da compilação.

Scout / Trivy

Scanners. Antes de seguir adiante, verificam imagens de contentores em busca de vulnerabilidades (CVEs) de terceiros.

Fase 4: Teste (Test)

Garante que os novos recursos funcionem sem quebrar o sistema. A Verificação de Segurança aqui é obrigatória, pois achar falhas nesta etapa é muito mais barato do que em produção.

Testes Automatizados

JUnit

Unidade: Framework para Java que valida pedaços isolados de código, garantindo resultados corretos.

Selenium

Automatização Web: Simula as ações de um utilizador real no navegador para testar fluxos completos e repetitivos.

Playwright

E2E Moderno: Da Microsoft, oferece testes de ponta a ponta ultra rápidos e fiáveis em múltiplos navegadores.

Fase 5: Liberação (Release)

A Ponte Dev-Ops

Foco na Integração e Entrega Contínuas (CI/CD). O software validado é empacotado automaticamente. A intervenção humana é minimizada para evitar erros.

Motores de CI/CD

GitLab CI

Integra o controlo de versões com um motor dinâmico e seguro.

GitHub Actions

Nativo do GitHub, automatiza workflows de forma muito acessível e visual para a comunidade.

Jenkins

Servidor de automatização open source global com milhares de plugins.

Fase 6: Implantação (Deploy)

Aceleração controlada. Provisionando servidores em segundos e distribuindo a aplicação validada para o público final sem causar indisponibilidades operacionais.

Infraestrutura como Código (IaC)

Provedores de Nuvem

AWS, GCP, Azure fornecem o hardware virtual e elástico escalando conforme a procura.

Terraform

Ferramenta agnóstica para declarar e provisionar de forma segura toda a infraestrutura via scripts.

Chef & Puppet

Gestão de configuração que mantém servidores com o estado exato exigido.

Fase 7: Operação (Operate)

A aplicação está agora "viva" e a receber interações de utilizadores reais em escala de produção.

As operações garantem disponibilidade, mitigando riscos de indisponibilidade ou invasão através da observabilidade.

O DevSecOps extingue a barreira antiga: os programadores passam a ter visibilidade do sistema.

Ferramentas de Observabilidade

Splunk

Capta massas de logs e gera alertas de segurança vitais.

Datadog

Unifica APM (rastreo), logs e infraestrutura.

Prometheus & Grafana

Recolha ativa de métricas combinada com painéis visuais poderosos.

Fase 8: Monitorização (Monitor)

O Início do Novo Fim

Vigiamos continuamente a saúde da infraestrutura e procuramos proativamente por ameaças.

Toda a inteligência e anomalias recolhidas não param aqui: elas retornam à Fase 1 (Planeamento), ensinando a equipa a construir um sistema blindado na próxima iteração do ciclo.



"O DevSecOps foca na construção de processos repetíveis e abertos para criar uma cultura. A tecnologia deve ajudar a concluir o trabalho, mas a ferramenta não define o trabalho."

Obrigado pela Leitura!

Produzido por: Marlon Bastida